

Data Structures & Algorithms - Complete Guide

A comprehensive DSA practice repository covering all essential topics in **C++**, **Python**, and **Java**.

Folder Structure

```
DSA/
├── 01-Arrays/           ├── cpp/ | python/ | java/
├── 02-HashMap/          ├── cpp/ | python/ | java/
├── 03-LinkedList/       ├── cpp/ | python/ | java/
├── 04-Stack/            ├── cpp/ | python/ | java/
├── 05-BinaryTree/       ├── cpp/ | python/ | java/
├── 06-Graph/            ├── cpp/ | python/ | java/
├── 07-Heap/             ├── cpp/ | python/ | java/
├── 08-BinarySearch/     ├── cpp/ | python/ | java/
├── 09-Backtracking/     ├── cpp/ | python/ | java/
├── 10-DynamicProgramming/ ├── cpp/ | python/ | java/
├── 11-BitManipulation/  ├── cpp/ | python/ | java/
├── 12-Matrix/           ├── cpp/ | python/ | java/
├── 13-DataStructures/   ├── cpp/ | python/ | java/
└── 14-SortingAlgorithms/ ├── cpp/ | python/ | java/
```

Topic Overview

1. Arrays

Difficulty	☒ Easy to ☐ Medium
Description	Contiguous memory data structure storing elements of the same type. The foundation of almost all DSA problems.
What to Learn	Traversal, insertion, deletion, two-pointer technique, sliding window, prefix sums, kadane's algorithm, subarray problems, rotation, merging intervals
Key Patterns	Two Pointers, Sliding Window, Prefix Sum
Prerequisites	None — start here

2. Hash Map

Difficulty	☒ Easy to ☐ Medium
Description	Key-value data structure providing $O(1)$ average-case lookup, insertion, and deletion via hashing.

What to Learn	Hash functions, collision handling (chaining, open addressing), frequency counting, two-sum patterns, grouping/bucketing, anagram detection, LRU cache design
Key Patterns	Frequency Map, Complement Lookup, Grouping
Prerequisites	Arrays

3. Linked List

Difficulty	🟡 Easy to 🟠 Medium
Description	Linear data structure where elements (nodes) are connected via pointers. Supports singly, doubly, and circular variants.
What to Learn	Traversal, reversal, cycle detection (Floyd's algorithm), merging sorted lists, finding middle node, intersection detection, LRU cache implementation
Key Patterns	Fast & Slow Pointers, Dummy Head, In-place Reversal
Prerequisites	Arrays, Pointers/References

4. Stack

Difficulty	🟡 Easy to 🟠 Medium
Description	LIFO (Last-In-First-Out) data structure. Fundamental for expression evaluation, parsing, and backtracking scenarios.
What to Learn	Push/pop operations, balanced parentheses, next greater/smaller element, monotonic stack, expression evaluation (infix/postfix), min stack, stock span problems
Key Patterns	Monotonic Stack, Nested Structure Parsing, State Tracking
Prerequisites	Arrays, Linked List

5. Binary Tree

Difficulty	🟠 Medium to 🟢 Hard
Description	Hierarchical data structure where each node has at most two children. Includes BST, AVL, and segment trees.
What to Learn	Traversals (inorder, preorder, postorder, level-order), BST operations, tree construction from traversals, LCA, diameter, balanced tree checks, serialization/deserialization
Key Patterns	DFS (recursive), BFS (level-order), Path Sum, Tree Construction
Prerequisites	Recursion, Stack, Queue

6. Graph

Difficulty	🕒 Medium to 🕒 Hard
Description	Non-linear data structure consisting of vertices and edges. Models relationships and networks.
What to Learn	BFS, DFS, topological sort, Dijkstra's, Bellman-Ford, Floyd-Warshall, union-find, Kruskal's/Prim's MST, cycle detection, connected components, bipartite check
Key Patterns	BFS/DFS Traversal, Shortest Path, Union-Find, Topological Sort
Prerequisites	Arrays, HashMap, Queue, Recursion

7. Heap (Priority Queue)

Difficulty	🕒 Medium
Description	Complete binary tree satisfying the heap property (min-heap or max-heap). Efficient for finding min/max elements.
What to Learn	Heapify, insert, extract-min/max, top-K elements, merge K sorted lists, median maintenance, task scheduling, heap sort
Key Patterns	Top-K, Two Heaps (median), Merge K Sorted, Greedy with Heap
Prerequisites	Arrays, Binary Tree basics

8. Binary Search

Difficulty	🕒 Easy to 🕒 Medium
Description	Divide-and-conquer search algorithm on sorted data with $O(\log n)$ complexity. Extends beyond simple search to optimization problems.
What to Learn	Standard binary search, search in rotated array, finding boundaries (lower/upper bound), binary search on answer, peak finding, search in 2D matrix
Key Patterns	Search Space Reduction, Binary Search on Answer, Boundary Finding
Prerequisites	Arrays, Sorting

9. Backtracking

Difficulty	🕒 Medium to 🕒 Hard
Description	Algorithmic technique that incrementally builds candidates and abandons (backtracks) candidates that fail to satisfy constraints.

What to Learn	Permutations, combinations, subsets, N-Queens, Sudoku solver, word search, palindrome partitioning, constraint satisfaction problems
Key Patterns	Choose-Explore-Unchoose, Pruning, State Space Tree
Prerequisites	Recursion, Arrays

10. Dynamic Programming

Difficulty	🕒 Medium to 🕒 Hard
Description	Optimization technique solving complex problems by breaking them into overlapping subproblems and storing results to avoid recomputation.
What to Learn	Memoization (top-down), tabulation (bottom-up), 1D/2D DP, knapsack (0/1, unbounded), LCS, LIS, edit distance, coin change, matrix chain multiplication, DP on trees/graphs
Key Patterns	Optimal Substructure, Overlapping Subproblems, State Transition
Prerequisites	Recursion, Arrays, Backtracking

11. Bit Manipulation

Difficulty	🕒 Medium
Description	Directly manipulating bits of numbers using bitwise operators. Enables highly optimized solutions for certain problems.
What to Learn	AND, OR, XOR, NOT, shifts, check/set/clear bits, power of two, counting set bits, single number problems, subsets via bitmask, XOR tricks
Key Patterns	XOR Properties, Bitmask DP, Bit Counting
Prerequisites	Binary number system, Arrays

12. Matrix

Difficulty	🕒 Medium
Description	2D array problems involving grid traversal, transformation, and search. Common in interviews for spatial reasoning.
What to Learn	Spiral traversal, rotation (90°/180°), search in sorted matrix, island counting (DFS/BFS on grid), path finding, dynamic programming on grids, diagonal traversal
Key Patterns	Direction Arrays, BFS/DFS on Grid, Layer-by-Layer
Prerequisites	Arrays, BFS/DFS, Dynamic Programming basics

13. Data Structures (Advanced)

Difficulty	🟡 Medium to 🟢 Hard
Description	Implementation and usage of advanced data structures beyond the basics — Tries, Segment Trees, Fenwick Trees, Disjoint Set Union, etc.
What to Learn	Trie (prefix tree), Segment Tree (range queries), Fenwick/BIT, Disjoint Set Union (Union-Find), LRU/LFU Cache, Deque, Monotonic Queue, Skip List
Key Patterns	Range Queries, Prefix Matching, Disjoint Sets
Prerequisites	Arrays, Trees, Graphs

14. Sorting Algorithms

Difficulty	🟡 Easy to 🟡 Medium
Description	Algorithms for arranging elements in a specific order. Understanding time/space tradeoffs and stability is crucial.
What to Learn	Bubble, Selection, Insertion, Merge Sort, Quick Sort, Heap Sort, Counting Sort, Radix Sort, Bucket Sort, Tim Sort — stability, time/space complexity analysis
Key Patterns	Divide & Conquer, Comparison vs Non-comparison, Stability
Prerequisites	Arrays, Recursion

Suggested Learning Path

```
Phase 1 (Foundation):    Arrays → HashMap → Sorting → Binary Search
Phase 2 (Linear DS):     Linked List → Stack → Matrix
Phase 3 (Non-Linear DS): Binary Tree → Heap → Graph
Phase 4 (Techniques):    Backtracking → Dynamic Programming → Bit Manipulation
Phase 5 (Advanced):      Data Structures (Trie, Segment Tree, Union-Find)
```

File Naming Convention

Use descriptive names with problem context:

```
cpp/    → two_sum.cpp, sliding_window_max.cpp
python/ → two_sum.py, sliding_window_max.py
java/   → TwoSum.java, SlidingWindowMax.java
```

Difficulty Legend

Symbol	Level	Description
	Easy	Fundamental concepts, direct implementation
	Medium	Requires combining patterns, moderate complexity
	Hard	Complex logic, multiple techniques, optimization needed